TEE(Trusted Execution Environment)에서 프로그램 실행하기

목차

1. TEE 개요
2. dstack 아키텍처
3. 프로그램 실행 환경 구성
4. 컨테이너 기반 애플리케이션 배포
5. 보안 기능 및 증명
6. 키 관리 및 비밀 데이터 처리
7. 네트워킹 및 외부 접근
8. 모니터링 및 로깅
9. 고급 기능
10. 문제 해결

1. TEE 개요

1.1 TEE란 무엇인가?

TEE(Trusted Execution Environment)는 메인 프로세서에서 격리된 보안 영역으로, 민감한 코드와 데이터를 보호하는 환경입니다. Intel TDX(Trust Domain Extensions)는 하드웨어 기반 TEE 구현 중 하나입니다.

1.2 dstack의 TEE 활용

dstack은 TEE 환경에서 컨테이너화된 애플리케이션을 쉽게 배포하고 관리할 수 있는 개발자 친화적인 플랫폼입니다.

주요 특징:

- 🔒 **보안 배포**: TEE 환경에서 컨테이너 앱을 몇 분 만에 안전하게 배포
- 🛠️ **친숙한 도구**: docker-compose.yaml 파일을 사용한 익숙한 배포 방식
- 🔑 **비밀 관리**: 민감한 데이터와 비밀 정보의 안전한 관리
- 📡 **자동 TLS**: 자동화된 TLS 종료를 통한 서비스 노출

---

2. dstack 아키텍처

2.1 핵심 컴포넌트

```
┌─────────────────┐  ┌─────────────────┐  ┌─────────────────┐
│    dstack-vmm   │ │  dstack-gateway │ │    dstack-kms   │
│    (VM 관리자)   │ │  (리버스 프록시) │ │   (키 관리 서버)  │
└─────────────────┘  └─────────────────┘  └─────────────────┘
│                       │                       │
└───────────────────────┼───────────────────────┘
                        │
                  ┌─────────────────┐
                  │ dstack-guest-   │
                  │ agent           │
```

```
                    | (CVM 내부 서비스)|

                    └──────────────────┘
```

2.1.1 dstack-vmm (Virtual Machine Manager)

- **역할**: TDX 호스트에서 CVM(Confidential Virtual Machine) 관리
- **기능**:
- VM 생성, 시작, 중지, 삭제
- 리소스 할당 및 관리
- 포트 매핑 및 네트워크 구성
- 웹 대시보드 제공

2.1.2 dstack-gateway (Gateway)

- **역할**: TLS 연결을 CVM으로 전달하는 리버스 프록시
- **기능**:
- 자동 TLS 인증서 관리
- 로드 밸런싱
- 도메인 기반 라우팅
- Zero Trust HTTPS 지원

2.1.3 dstack-kms (Key Management Service)

- **역할**: CVM용 키 생성 및 관리
- **기능**:
- 암호화 키 생성 및 배포
- 애플리케이션 인증

- 증명(Attestation) 검증

2.1.4 dstack-guest-agent

- **역할**: CVM 내부에서 실행되는 서비스
- **기능**:
- 컨테이너 키 유도 및 증명 요청 처리
- Docker 컨테이너 관리
- 시스템 모니터링

---

3. 프로그램 실행 환경 구성

3.1 시스템 요구사항

하드웨어 요구사항:

- Intel TDX 지원 베어메탈 서버
- 최소 16GB RAM
- 100GB 이상의 여유 디스크 공간
- 공인 IPv4 주소

소프트웨어 요구사항:

- Ubuntu 24.04 (권장)
- TDX 설정 완료
- Rust 개발 환경
- Docker 및 관련 도구

3.2 환경 설정

3.2.1 의존성 설치

```
# Ubuntu 24.04용 필수 패키지 설치
sudo apt install -y build-essential chrpath diffstat lz4 wireguard-tools xorriso

# Rust 설치
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
source ~/.cargo/env
```

3.2.2 dstack 빌드

```
# 저장소 클론
git clone https://github.com/Dstack-TEE/meta-dstack.git --recursive
cd meta-dstack/

# 빌드 디렉토리 생성 및 설정
mkdir build
cd build
../build.sh hostcfg

# 설정 파일 편집
vim ./build-config.sh
```

```
# 빌드 실행

../build.sh hostcfg
```

3.2.3 게스트 이미지 준비

```
# 옵션 A: 미리 빌드된 이미지 다운로드 (권장)

../build.sh dl 0.5.2


# 옵션 B: 소스에서 빌드 (시간이 오래 걸림)

../build.sh guest
```

---

4. 컨테이너 기반 애플리케이션 배포

4.1 기본 배포 과정

4.1.1 컴포넌트 시작

```
# 3개의 터미널에서 각각 실행

# 터미널 1: KMS 시작

./dstack-kms -c kms.toml


# 터미널 2: Gateway 시작 (관리자 권한 필요)

sudo ./dstack-gateway -c gateway.toml
```

```
# 터미널 3: VMM 시작

./dstack-vmm -c vmm.toml
```

4.1.2 웹 대시보드 접근

- VMM 대시보드: http://localhost:9080
- Gateway 대시보드: https://localhost:9070

4.2 애플리케이션 배포

4.2.1 docker-compose.yaml 작성

```yaml
version: '3'

services:

nginx:

image: nginx:latest

ports:

- "8080:80"

environment:

- NGINX_HOST=localhost

- NGINX_PORT=80

volumes:

- /var/run/dstack.sock:/var/run/dstack.sock

restart: always
```

4.2.2 배포 과정

1. VMM 웹 인터페이스에서 docker-compose.yaml 업로드
2. 애플리케이션 ID 설정
3. 리소스 할당 (메모리, CPU)
4. 배포 실행

4.2.3 배포 상태 확인

```
# CVM 로그 확인
tail -f run/vm/{vm-id}/serial.log


# 컨테이너 상태 확인 (CVM 내부)
ssh root@{cvm-ip}
docker ps
```

---

5. 보안 기능 및 증명

5.1 TDX Quote 획득

TEE 환경에서 실행 중임을 증명하기 위해 TDX Quote를 획득할 수 있습니다.

5.1.1 컨테이너에서 Quote 요청

```
# dstack 소켓을 통한 Quote 요청
curl --unix-socket /var/run/dstack.sock \
"http://localhost/GetQuote?report_data=0x1234deadbeef" | jq .
```

5.1.2 Quote 응답 예시

```
{
"quote": "base64_encoded_quote_data",
"report_data": "sha256_hash_of_input",
"timestamp": "2024-01-01T00:00:00Z"
}
```

5.2 증명 검증 과정

```rust
// Rust 코드 예시: Quote 검증
use tdx_attest::TdxQuote;

fn verify_quote(quote_data: &[u8]) -> Result<bool, Error> {
let quote = TdxQuote::parse(quote_data)?;
// 1. Quote 서명 검증
quote.verify_signature()?;
// 2. 측정값 검증
let expected_mr_td = get_expected_measurement();
if quote.td_report.mr_td != expected_mr_td {
return Err(Error::MeasurementMismatch);
}
// 3. 추가 검증 로직
Ok(true)
```

```
}
```

5.3 보안 부팅 과정

5.3.1 측정 과정

1. **TDVF (TDX Virtual Firmware)**: 부트로더 측정
2. **커널 측정**: Linux 커널 및 initrd 측정
3. **애플리케이션 측정**: 컨테이너 이미지 측정

5.3.2 RTMR (Runtime Measurement Register) 확장

```
# RTMR3에 사용자 정의 측정값 추가
echo "custom_measurement_data" | sha256sum | \
xxd -r -p > /sys/kernel/security/tdx_guest/rtmr3
```

---

6. 키 관리 및 비밀 데이터 처리

6.1 암호화된 환경 변수

6.1.1 비밀 데이터 전달

```
# docker-compose.yaml에서 암호화된 환경 변수 사용
version: '3'
services:
app:
```

```
image: myapp:latest
environment:
- DATABASE_PASSWORD=${ENCRYPTED_DB_PASSWORD}
- API_KEY=${ENCRYPTED_API_KEY}
```

6.1.2 클라이언트 측 암호화

```
// 웹 인터페이스에서 비밀 데이터 암호화
const encryptedData = await encryptForTEE(secretData, teePublicKey);
```

6.2 키 유도 과정

6.2.1 애플리케이션 키 요청

```
# CVM 내부에서 애플리케이션별 키 요청
curl -X POST http://localhost:9205/derive_key \
-H "Content-Type: application/json" \
-d '{
"app_id": "my-application",
"key_type": "encryption",
"context": "database_encryption"
}'
```

6.2.2 키 유도 응답

```
{
"key": "derived_key_base64",
"key_id": "unique_key_identifier",
"algorithm": "AES-256-GCM"
}
```

6.3 보안 저장소

6.3.1 임시 키 저장

```rust
// Rust에서 메모리 내 키 관리
use zeroize::Zeroize;

struct SecureKey {
data: Vec<u8>,
}

impl Drop for SecureKey {
fn drop(&mut self) {
self.data.zeroize();
}
}
```

7. 네트워킹 및 외부 접근

7.1 포트 매핑 및 라우팅

7.1.1 기본 라우팅 규칙

- `<id>[s].<base_domain>` → 포트 80 또는 443 (s가 있으면 443)
- `<id>-<port>[s].<base_domain>` → 지정된 포트

7.1.2 예시

```
# 예시 도메인 매핑
3327603e03f5bd1f-8080.app.example.com → CVM의 8080 포트
3327603e03f5bd1f-8080s.app.example.com → CVM의 8080 포트 (TLS 패스스루)
```

7.2 WireGuard VPN 설정

7.2.1 CVM 네트워크 구성

```
# CVM 내부 IP 확인
ip addr show wg0
```

```
# Gateway에서 WireGuard 상태 확인
wg show
```

7.2.2 네트워크 격리

- 각 CVM은 독립적인 네트워크 네임스페이스

- WireGuard를 통한 암호화된 통신
- 호스트와 격리된 네트워크 스택

7.3 TLS 설정

7.3.1 자동 인증서 관리

```
# gateway.toml 설정
[tls]
auto_cert = true
cert_path = "/path/to/cert.pem"
key_path = "/path/to/key.pem"

[acme]
directory_url = "https://acme-v02.api.letsencrypt.org/directory"
contact = ["mailto:admin@example.com"]
```

7.3.2 사용자 정의 도메인

```
# docker-compose.yaml에서 사용자 정의 도메인 설정
version: '3'
services:
app:
image: myapp:latest
labels:
```

```
- "dstack.domain=custom.example.com"

- "dstack.tls.passthrough=true"
```

---

8. 모니터링 및 로깅

8.1 컨테이너 로그 수집

8.1.1 로그 API 사용

```
# 실시간 로그 스트리밍

curl 'http://<appid>.app.example.com:9090/logs/<container_name>?follow=true&text



# 특정 시간 범위 로그

curl 'http://<appid>.app.example.com:9090/logs/<container_name>?since=1640995200
```

8.1.2 로그 파라미터

| 파라미터 | 설명 |

|----------|------|

| `since` | 시작 Unix 타임스탬프 |

| `until` | 종료 Unix 타임스탬프 |

| `follow` | 실시간 로그 스트리밍 |

| `text` | 텍스트 형식 반환 |

| `timestamps` | 타임스탬프 포함 |

| `bare` | JSON 래핑 없이 원시 로그 |

8.2 시스템 모니터링

8.2.1 리소스 사용량 확인

```
# CVM 내부에서 리소스 모니터링
htop
free -h
df -h

# 컨테이너별 리소스 사용량
docker stats
```

8.2.2 네트워크 모니터링

```
# 네트워크 연결 상태
ss -tulpn

# WireGuard 통계
wg show all dump
```

8.3 대시보드 모니터링

8.3.1 Guest Agent 대시보드

- URL: `http://<cvm-ip>:9190`

- 컨테이너 상태 및 로그
- 시스템 리소스 정보
- 네트워크 구성 정보

8.3.2 Gateway 대시보드

- URL: `https://localhost:9070`
- CVM 연결 상태
- 트래픽 통계
- 인증서 상태

---

9. 고급 기능

9.1 Zero Trust HTTPS

9.1.1 CAA 레코드 설정

```
# Cloudflare API를 통한 CAA 레코드 자동 설정
./certbot renew -c certbot.toml
```

9.1.2 Certificate Transparency 모니터링

```
# CT 로그 모니터링 시작
./ct_monitor -t https://localhost:9010/prpc -d app.example.com
```

9.2 멀티 노드 클러스터

9.2.1 클러스터 구성

```
# gateway.toml - 부트노드 설정
[cluster]
my_url = "https://gateway1.example.com:9202"
bootnode_url = "https://gateway1.example.com:9202"


# gateway.toml - 추가 노드 설정
[cluster]
my_url = "https://gateway2.example.com:9202"
bootnode_url = "https://gateway1.example.com:9202"
```

9.2.2 로드 밸런싱

- 자동 노드 발견
- 헬스 체크 기반 라우팅
- 장애 조치 지원

9.3 사용자 정의 KMS 구현

9.3.1 IAppAuth 인터페이스 구현

```
// Solidity 스마트 컨트랙트 예시
contract CustomAppAuth is IAppAuth {
function checkPermission(
bytes32 appId,
```

```
bytes32 composeHash,

bytes calldata deviceId

) external view returns (bool) {

// 사용자 정의 권한 검증 로직

return customAuthLogic(appId, composeHash, deviceId);

}

}
```

9.3.2 온체인 등록

```
# 사용자 정의 앱 컨트랙트 배포

npx hardhat app:deploy --contract CustomAppAuth --network phala



# KMS에 등록

npx hardhat kms:register-app --app-address 0x... --network phala
```

---

10. 문제 해결

10.1 일반적인 오류

10.1.1 CID 충돌 오류

```
Error: qemu-system-x86_64: vhost-vsock: unable to set guest cid: Address already
```

해결 방법:

```
# 사용 중인 CID 확인
ps aux | grep 'guest-cid='

# vmm.toml에서 CID 범위 변경
[cvm]
cid_start = 33000
cid_pool_size = 1000
```

10.1.2 권한 오류 (Ubuntu 23.10+)

```
PermissionError: [Errno 1] Operation not permitted
```

해결 방법:

```
# 사용자 네임스페이스 제한 해제
sudo sysctl kernel.apparmor_restrict_unprivileged_userns=0
```

10.2 네트워크 문제

10.2.1 포트 접근 불가

```
# 방화벽 설정 확인
sudo ufw status
```

```
# 포트 매핑 확인
netstat -tulpn | grep :9080
```

10.2.2 TLS 인증서 문제

```
# 인증서 상태 확인
openssl x509 -in cert.pem -text -noout

# Let's Encrypt 갱신
./certbot renew -c certbot.toml
```

10.3 성능 최적화

10.3.1 메모리 할당 최적화

```
# vmm.toml - CVM 메모리 설정
[cvm]
default_memory = "4G"
max_memory = "16G"
```

10.3.2 디스크 I/O 최적화

```
# SSD 사용 권장
# I/O 스케줄러 설정
```

```
echo mq-deadline > /sys/block/sda/queue/scheduler
```

10.4 디버깅 도구

10.4.1 로그 레벨 설정

```
# 상세 로그 출력
RUST_LOG=debug ./dstack-vmm -c vmm.toml
```

10.4.2 TDX 상태 확인

```
# TDX 지원 확인
dmesg | grep -i tdx

# TDX 게스트 모듈 상태
lsmod | grep tdx
```

결론

dstack을 통한 TEE 환경에서의 프로그램 실행은 다음과 같은 장점을 제공합니다:

1. **보안성**: 하드웨어 기반 격리 환경
2. **편의성**: 기존 Docker 워크플로우 활용
3. **확장성**: 멀티 노드 클러스터 지원
4. **투명성**: 증명 가능한 실행 환경

이 가이드를 통해 TEE 환경에서 안전하고 효율적으로 애플리케이션을 배포하고 관리할 수 있습니다.